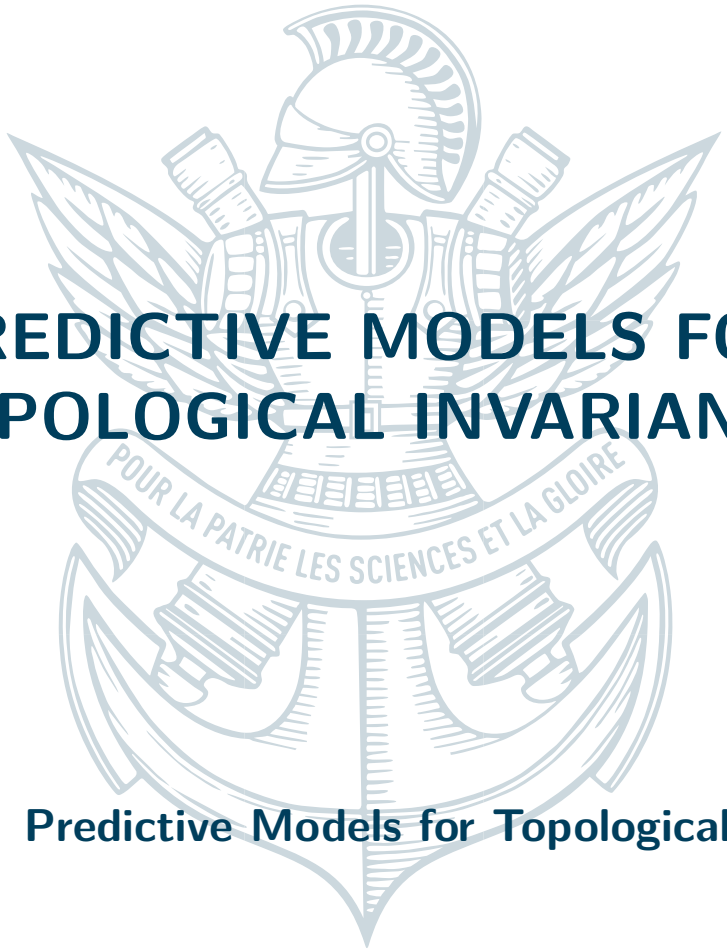


# PREDICTIVE MODELS FOR TOPOLOGICAL INVARIANTS

## Project 8 : Predictive Models for Topological Invariants

August 13, 2026

Ten Nguyen Hanaoka

**Abstract**

We study predictive models for topological invariants in Topological Data Analysis, with a focus on point-cloud learning. The central object is the persistence diagram and its vectorizations, which provide stable summaries of geometric structure but remain expensive to compute exactly. Motivated by RipsNet, which learns persistence-based descriptors directly from unordered point sets, we investigate a broader family of symmetry-aware models. The project developed in three stages: first `DistanceMatrixRaggedModel`, then `PersNet`, and finally a family of Tensor Field Network (TFN) architectures generalized to dimension $n$. This final step was motivated by the desire to obtain robustness to both point permutations and rigid isometries, which the earlier models did not fully provide. The main contribution of the paper is this TFN family for geometric learning on 3D and higher-dimensional point clouds, together with the two earlier baselines. A central methodological point is that rotation equivariance is not automatic: it must be enforced jointly by the basis functions, the Clebsch–Gordan couplings, and the nonlinearities, and it is easy to break each one of these pieces independently. We describe the implementation choices that guarantee equivariance—a fixed change of basis from raw coordinates to the vector feature, quadrature-based Clebsch–Gordan coefficients computed in the same basis as the geometric features, and nonlinearities that act on scalar channels only—and we verify numerically that the resulting models are invariant to random rotations of the input. We also describe the PyTorch implementation and experimental workflow used to compare these models on synthetic and UCR benchmarks under clean, noisy, and symmetry-perturbed settings.

# CONTENTS

# 1 INTRODUCTION

Topological Data Analysis studies data through topological invariants such as persistence diagrams. These descriptors are informative and stable, but their exact computation requires building filtered simplicial complexes and computing persistent homology, which can become expensive for large or dense point clouds.

Learning-based approaches seek to approximate these descriptors directly from the input geometry. RipsNet [14] is a representative example: it maps unordered point clouds to vectorized persistence descriptors using a permutation-invariant neural architecture. The project behind this paper originally began from a PyTorch reimplementation of that line of work and then expanded toward symmetry-aware geometric models that could better exploit the structure of point clouds.

The project developed in three stages. We first introduced `DistanceMatrixRaggedModel`, a persistence-oriented model based on pairwise distances. We then added `PersNet` as a permutation-invariant baseline. Finally, we generalized the TFN family to arbitrary ambient dimension $n$ in order to target robustness to both permutations and isometries more directly than before.

The resulting study therefore compares three families of models in their historical order of development. First comes `DistanceMatrixRaggedModel`. Second comes `PersNet`. Third comes the TFN family, which is the most symmetry-aware part of the project and the one that motivated the higher-dimensional generalization.

The empirical evaluation is carried out on two complementary testbeds. Synthetic point-cloud benchmarks probe whether the models recover topological information under noise and rigid transformations, while a suite of UCR datasets provides a broader benchmark for robustness and sample efficiency. Together, these experiments make it possible to study how different inductive biases behave when the input geometry, ordering, and noise level vary.

The remainder of the paper is organized as follows. Section 2 collects the mathematical preliminaries, Section 3 reviews related work, Section 4 presents the model families, Section 5 describes the experimental setup, and Section 6 discusses the results.

# 2 BACKGROUND

We collect the mathematical preliminaries used throughout the paper. The first part recalls persistent homology and the standard vectorizations of persistence diagrams. The second part recalls permutation-invariant set functions, group actions, and the representation-theoretic language used to describe symmetry-aware models on point clouds.

## 2.1 Persistence Diagrams

**Filtered complexes.** Let $(\mathcal{K}_t)_{t\in\mathbb{R}}$ be a filtration of simplicial complexes, meaning that $\mathcal{K}_s \subseteq \mathcal{K}_t$ whenever $s \leqslant t$. Applying homology in degree $k$ gives a persistence module $(H_k(\mathcal{K}_t))_{t\in\mathbb{R}}$. The associated persistence diagram is the multiset

$$D_k = \{(b_i, d_i)\}_{i=1}^m \subset \{(b, d) \in \mathbb{R}^2 : b < d\},$$

where each point records the birth and death times of a homology class.

**Vietoris-Rips filtration.** For a point cloud $\mathcal{P} = \{p_1, \ldots, p_n\} \subset \mathbb{R}^d$, the Vietoris-Rips complex at scale $t$ is

$$\mathrm{VR}_t(\mathcal{P}) = \{\sigma \subseteq \mathcal{P} : \|p_i - p_j\|_2 \leqslant t \text{ for all } p_i, p_j \in \sigma\}.$$

Equivalently, if $d(x, \mathcal{P}) = \min_{p\in\mathcal{P}} \|x - p\|_2$, then the sublevel sets of $d(\cdot, \mathcal{P})$ generate a filtration by unions of balls. Because $\mathrm{VR}_t(\mathcal{P})$ depends only on pairwise distances, it is invariant under rigid motions of $\mathbb{R}^d$.

## 2.2 Bottleneck Distance and Stability

The bottleneck distance between two persistence diagrams $D_1$ and $D_2$ is

$$W_\infty(D_1, D_2) = \inf_{\gamma} \sup_{x\in D_1} \|x - \gamma(x)\|_\infty,$$

where $\gamma$ ranges over bijections between $D_1 \cup \Delta$ and $D_2 \cup \Delta$, with $\Delta = \{(t, t) : t \in \mathbb{R}\}$ the diagonal. Informally, one is allowed to match points to each other or to the diagonal.

The fundamental stability statement is that small perturbations of the input induce small perturbations of the diagram. For tame functions $f, g$ on the same domain, one has the estimate

$$W_\infty(D_k(f), D_k(g)) \leqslant \|f - g\|_\infty.$$

This stability property is one of the main reasons persistence-based descriptors are useful in learning.

## 2.3 Vectorizations of Persistence Diagrams

Persistence diagrams live in a nonlinear space, so they are often mapped to finite-dimensional vectors. Two standard constructions are persistence images and persistence landscapes.

**Persistence images.** Let $D = \{(b_i, d_i)\}_{i=1}^m$. After the change of variables $(b, d) \mapsto (b, d - b)$, a weighted persistence surface can be written as

$$\rho_D(x, y) = \sum_{i=1}^{m} w(b_i, d_i)\, K_\sigma((x, y) - (b_i, d_i - b_i)),$$

where $w$ is a weight function and $K_\sigma$ is a Gaussian kernel. The persistence image is obtained by integrating $\rho_D$ over a fixed grid of pixels.

**Persistence landscapes.** For each point $(b, d) \in D$, define the tent function

$$\Lambda_{(b,d)}(t) = \max\{0, \min(t - b, d - t)\}.$$

The persistence landscape is the sequence $\lambda_k(t)$, where $\lambda_k(t)$ is the $k$-th largest value among the set $\{\Lambda_{(b_i,d_i)}(t)\}_{i=1}^m$ at each $t \in \mathbb{R}$. Sampling the functions $\lambda_k$ on a grid gives a finite vector representation.

## 2.4 Permutation-Invariant Set Functions

A finite point cloud can be viewed as an unordered multiset

$$X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d.$$

A function $F$ on such inputs is permutation-invariant if $F(X) = F(\pi X)$ for every permutation $\pi \in S_n$. The Deep Sets theorem shows that many continuous invariant functions admit the form

$$F(X) = \rho\left(\sum_{i=1}^{n} \phi(x_i)\right),$$

for suitable maps $\phi$ and $\rho$ [1]. The sum can be replaced by another symmetric reduction, such as max pooling, when one wants a different aggregation rule; this is the structural idea behind PointNet [2]. In practice, ragged batches are implemented by masking padded entries before the symmetric pooling step. This same viewpoint also explains why a row-wise encoder applied to a pairwise distance matrix can still produce a permutation-invariant prediction after symmetric aggregation.

## 2.5 Group Actions and Equivariance

Let $G$ be a group acting on a set $X$, and let $\rho : G \to GL(V)$ be a linear representation on a vector space $V$. A map $F : X \to V$ is *equivariant* if

$$F(g \cdot x) = \rho(g)F(x) \qquad \text{for all } g \in G,\ x \in X.$$

It is *invariant* if $\rho(g) = I_V$ for every $g \in G$.

For point clouds, a second natural symmetry is permutation of the input ordering. If $\pi \in S_n$ acts by reindexing the points of $P = (p_1, \ldots, p_n)$, then a permutation-invariant model satisfies

$$F(p_1, \ldots, p_n) = F(p_{\pi(1)}, \ldots, p_{\pi(n)}).$$

## 2.6 Representations and Tensor Products

Many symmetry-aware models organize their hidden states according to representation type. A finite-dimensional representation of a group $G$ is a vector space $V$ together with a homomorphism $\rho : G \to GL(V)$. When $V$ is reducible, it decomposes as a direct sum of irreducible representations:

$$V \cong \bigoplus_{\lambda} m_\lambda V_\lambda.$$

For $SO(3)$, the irreducible representations are indexed by $\ell \in \mathbb{N}$ and have dimension $2\ell + 1$. Their tensor products decompose according to

$$V_{\ell_1} \otimes V_{\ell_2} \cong \bigoplus_{\ell=|\ell_1-\ell_2|}^{\ell_1+\ell_2} V_\ell.$$

This decomposition is the algebraic reason tensor features can be coupled while preserving symmetry. For higher-dimensional ambient spaces, the same idea applies after replacing $SO(3)$ by the symmetry group appropriate to the problem, typically $SO(n)$ or $O(n)$. The irreducible blocks and their bases then depend on $n$, but the guiding principle remains the same: decompose tensor products into symmetry-adapted components and keep track of the corresponding intertwiners.

## 2.7 Clebsch-Gordan Coefficients

Clebsch-Gordan coefficients are the change-of-basis constants associated with the above decomposition. If $\{e^{\ell_1}_{m_1}\}$, $\{e^{\ell_2}_{m_2}\}$, and $\{e^{\ell}_{m}\}$ are orthonormal bases of the corresponding irreducible spaces, then

$$e^{\ell_1}_{m_1} \otimes e^{\ell_2}_{m_2} = \sum_{\ell,m} C^{\ell_1,\ell_2,\ell}_{m_1,m_2,m}\, e^{\ell}_{m}.$$

The coefficients $C^{\ell_1,\ell_2,\ell}_{m_1,m_2,m}$ encode how two representations can be coupled into a third one. More generally, if

$$V_\alpha \otimes V_\beta \cong \bigoplus_{\gamma} N^{\gamma}_{\alpha\beta} V_\gamma,$$

then the Clebsch-Gordan coefficients are the entries of the change-of-basis map from the product basis to an adapted irreducible basis. For compact groups, one numerical route is to block-diagonalize the tensor-product representation and extract the intertwining basis [10]. For $SU(N)$ and $SL(N, \mathbb{C})$, the Gelfand-Tsetlin pattern calculus gives an explicit algorithm [11]. In higher-rank Cartesian tensor settings, recent decomposition methods provide scalable orthogonal bases for equivariant spaces and are especially relevant when one wants to generalize the construction from 3D to dimension $n$ [12].

## 2.8 Local Geometric Encodings

Let $P = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ be a point cloud. Its $k$-nearest-neighbor graph is

$$G_k(P) = (V, E), \qquad V = \{1, \dots, n\}, \qquad E = \{(i, j) : j \in \mathcal{N}_k(i)\}.$$

For each $i$, let $\mathcal{N}_k(i)$ denote the set of its $k$ nearest neighbors. For $j \in \mathcal{N}_k(i)$, define the relative vector

$$r_{ij} = x_j - x_i, \qquad d_{ij} = \|r_{ij}\|_2, \qquad \hat{r}_{ij} = \frac{r_{ij}}{\|r_{ij}\|_2}.$$

A radial basis expansion of the distance can be written as

$$\phi_m(d) = \exp\left(-\frac{(d-\mu_m)^2}{\sigma^2}\right), \qquad m = 1, \ldots, M,$$

for centers $\mu_m$ and width $\sigma > 0$. These radial and directional quantities form the local geometric data used by the models studied later in the paper. They are the basic ingredients of the graph-based message-passing layers in the TFN family, where reindexing the points only relabels the graph nodes.

# 3 RELATED WORK

Learning with topological features has developed along several complementary directions. Early work focused on turning persistence diagrams into vector spaces through constructions such as persistence images and persistence landscapes, making them compatible with standard neural architectures and downstream classifiers [5, 6]. The stability theorem for persistence diagrams provides the mathematical justification for these constructions [7], and later work studied perturbation robustness and optimized persistence-based losses and layers [15, 16, 17, 18].

Another line of research learns topological or geometric signatures directly from point clouds. RipsNet [14] is a representative example: it predicts vectorized persistence descriptors from unordered point sets using a Deep Sets-style architecture. In a similar spirit, the Deep Sets theorem [1] and PointNet [2] show how permutation invariance can be built into the architecture through shared pointwise encoders and symmetric pooling. Later point-cloud baselines such as PointNet++ [3] and DGCNN [4] improve on this idea by adding local neighborhoods and graph-based feature extraction.

Symmetry-aware geometric learning extends these ideas from permutation invariance to rigid-motion equivariance. Tensor Field Networks [8] are a central reference point in this literature because they couple point coordinates and tensor features through equivariant tensor products. More recent work on any-dimensional equivariant neural networks [9] provides a principled route for extending such constructions beyond the 3D setting, which is exactly the direction taken by the TFN variants developed in this project. The representation-theoretic tools needed for these models, including Clebsch-Gordan computations and high-rank tensor decompositions, are by now supported by explicit algorithms and decomposition frameworks [10, 11, 12].

Finally, the UCR archive [13] remains a standard benchmark suite for time-series classification and provides a useful complementary testbed for robustness and sample-efficiency comparisons. The recent survey literature on topological machine learning also offers broader context for where this project sits among topology-aware neural approaches [19, 20, 21, 22, 23].

# 4 MODELS

We present the models in the same progression as the project itself. The first model introduced was the distance-matrix approach, followed by the PointNet baseline, and finally the TFN family generalized to dimension $n$.

## 4.1 DistanceMatrixRaggedModel

`DistanceMatrixRaggedModel` is one of the new models introduced in this paper for persistence-diagram prediction. Instead of operating on raw coordinates, it uses pairwise distance matrices, which makes it directly tied to geometry and better aligned with the topology prediction task. Because the same row encoder is applied to every row and the outputs are pooled symmetrically, the model behaves like a set function on the rows of the distance matrix. This makes it natural to handle ragged batches while retaining permutation invariance after reindexing of the points.

**Algorithm 1:** DistanceMatrixRaggedModel Forward Pass

**Input:** Batch of pairwise distance matrices $\mathcal{D} = \{D_1, \ldots, D_B\}$.
**Output:** Predicted topological descriptors.
Process each row of each distance matrix with a shared MLP;
Aggregate row features with summation pooling;
Apply the final prediction head;
**return** descriptors;

## 4.2 PersNet

`PersNet` (for "Persistence"), included as a permutation-invariant baseline, is the RipsNet study's plain point-cloud model: it applies a shared MLP to each point and then aggregates the features by max pooling. This makes it a useful reference model, and it is the canonical PointNet/Deep Sets pattern [1, 2]. The architecture is efficient and robust to permutations, but it does not enforce rotation equivariance and it consumes raw coordinates only, with no persistence structure, so it isolates the effect of the geometric and topological machinery in the other models.

**Algorithm 2:** PersNet Forward Pass

**Input:** Batch of ragged point clouds $\mathcal{P} = \{P_1, \ldots, P_B\}$.
**Output:** Predicted descriptors or class scores.
Apply a shared MLP to each point;
Mask padded values;
Max-pool over points in each cloud;
Feed the pooled representation to the final MLP;
**return** predictions;

## 4.3 Tensor Field Networks

The `TensorFieldNetwork` class is the main public TFN interface. It implements an SO(3)-equivariant point-cloud classifier for 3D inputs, and it is the closest analogue of the original TFN formulation in the current codebase [8]. Its role in the paper is to serve as the canonical geometric baseline: the model is expressive enough to encode rotations, but still simple enough to interpret as a standard TFN. The higher-dimensional generalization in this project was motivated by a stronger symmetry goal: we wanted robustness to both permutations and rigid isometries, rather than only one of these symmetries at a time. In that sense, the $n$-dimensional TFN family is the main architectural contribution of the project.

At a schematic level, a TFN layer forms geometric messages from relative positions, couples them with feature tensors, and then projects the result onto irreducible channels using the relevant Clebsch-Gordan coefficients. One can think of the update as

$$m_{ij}^{(\ell)} = \Phi_\ell(d_{ij}) \otimes b_\ell(\hat{r}_{ij}), \qquad h_i^{(\ell+1)} = \sum_{j \in \mathcal{N}_k(i)} \Pi_\ell\left(h_j^{(\ell)} \otimes m_{ij}^{(\ell)}\right),$$

where $\Phi_\ell$ denotes a radial encoding, $b_\ell$ denotes a geometric basis, and $\Pi_\ell$ denotes the symmetry-adapted projection. This is the algebraic mechanism that makes the network equivariant.

**GTTensorFieldNetworkV2** The `GTTensorFieldNetworkV2` model is the most developed TFN variant in the repository. It generalizes the base TFN to arbitrary ambient dimension $n$ and adds the main implementation improvements:

- sparse k-nearest-neighbor neighborhoods instead of dense all-pairs interactions;
- gated nonlinearities, where scalar channels control the activation of higher-order features;
- residual projections per geometric type;
- channel mixing blocks between message-passing layers;
- optional node-attribute injection at the input.

This version is the most practical TFN variant when one wants a stable model that can scale beyond small toy clouds. It is also the version that most directly reflects the project's move from a 3D-only formulation to a more general $n$-dimensional symmetry-aware setting.

**Equivariance in practice** Enforcing rotation equivariance in the implementation requires care at several points that are easy to get wrong independently. We list the choices made in the codebase, all of which are verified by the tests reported in Section 6.6.

- **Basis self-consistency.** The relative direction $\hat{r}_{ij}$ is expanded in a real orthonormal basis of $S^{n-1}$: real spherical harmonics for $n = 3$, and the Gelfand–Tsetlin basis [11] for $n \geqslant 4$. The same basis is used for the edge features, the hidden tensor features, and the coupling coefficients, so that no change of basis is silently skipped between the geometric input and the equivariant tensor products.

- **Vector feature change of basis.** The first-order block of the input feature must literally be the coordinate direction $x$ of the point. Since the spherical-harmonic evaluation returns $Y_1(x)$ rather than $x$, the implementation recovers the fixed matrix $M$ with $Y_1(x) = Mx$ by a least-squares fit over random unit vectors (cached once at construction). Without this change of basis, the vector channel is misaligned with the edge harmonics and equivariance is broken for $n = 2$ and $n \geqslant 4$.
- **Clebsch–Gordan coefficients.** For $n \geqslant 2$, the coupling coefficients are computed by numerical quadrature of the triple product of basis functions over $S^{n-1}$, on a spherical grid fine enough to integrate polynomials up to degree $3\ell_{\max}$ exactly [10]. For $n = 3$, a closed-form real-valued Clebsch–Gordan table obtained from real-spherical-harmonic triple products is used instead. Both procedures yield couplings that are exactly consistent with the basis used everywhere else in the network, and the grids are cached so that the cost is paid once per configuration.
- **Gated nonlinearity.** The equivariant gate $g = \sigma(\text{linear}(f_0))$ is computed from the scalar channel only and applied identically to every component of a higher-order block. Scalar channels are processed by a nonlinear channel MLP; non-scalar blocks are mixed by bias-free linear maps. A blockwise nonlinearity applied to $l > 0$ components would break equivariance, which is the single most common cause of the bugs fixed during this project.
- **Equivariant pooling and sampling.** Hierarchical pooling uses attention-weighted sums instead of per-component max pooling (which is not equivariant), and farthest-point sampling uses a deterministic seed so that the clustering step is a deterministic function of the cloud.

**HierarchicalGTTFN** The `HierarchicalGTTFN` model is a multi-scale version of the TFN family. Rather than processing the full point cloud at a single resolution, it builds intermediate geometric summaries and pools them hierarchically. This is useful when the point clouds become larger and a single global pass becomes expensive.

**OnEquivariantTensorFieldNetwork** The `OnEquivariantTensorFieldNetwork` variant extends the symmetry group from rotations to orthogonal transformations. It is therefore designed to be insensitive not only to rotations, but also to reflections. In practice, this variant is attractive when the task should depend on intrinsic geometry but not on orientation parity.

**Common design** Despite their differences, these TFN variants share the same core idea:

- build features from distances and directions;
- propagate information through equivariant tensor products;
- preserve geometric structure while learning expressive point-cloud representations.

**Algorithm 3:** TFN Layer Overview

**Input:** Point cloud $P \in \mathbb{R}^{N \times n}$ and neighborhood size $k$.
**Output:** Updated geometric features.
Build a k-NN graph on the point cloud;
Encode pairwise distances with an RBF expansion;
Compute directional geometric features from relative vectors;
Propagate messages through equivariant tensor products;
Apply gating, residual connections, and channel mixing;
**return** updated features;

At a high level, the TFN variants differ mainly in how much structure they add around this core layer: the base `TensorFieldNetwork` keeps the architecture close to the original TFN idea, while `GTTensorFieldNetworkV2` adds the most aggressive engineering improvements, and the hierarchical / orthogonal variants adapt the same backbone to larger or more symmetry-sensitive settings.

# 5 EXPERIMENTAL SETUP

The experiments are organized around three complementary settings: synthetic point-cloud tasks, UCR time-series benchmarks recast as point-cloud problems, and notebook-driven exploratory analyses. The common objective is to compare geometric inductive biases under clean and perturbed inputs.

## 5.1 Evaluation Protocol

We evaluate the models using three metrics:

- classification accuracy on clean and noisy data;
- isometry robustness, measured by the variation of the output under rigid transformations;
- permutation robustness, measured by the variation of the output under point reordering.

The TFN models are expected to perform best on isometry-related tests because they build geometric symmetry into the architecture. PointNet-style models are expected to be robust to permutations. DistanceMatrixRaggedModel benefits from the geometric prior induced by pairwise distances, which is particularly relevant for persistence-oriented tasks.

## 5.2 Synthetic Point-Cloud Benchmarks

The synthetic experiments are the most direct way to study topological prediction. They use generated point clouds in which the target depends on the underlying shape, such as the number of circles or the presence of topological loops. These tasks are useful because they isolate the geometric part of the learning problem: the model must infer topology from noisy point samples rather than from hand-crafted features.

In the earlier stages of the project, synthetic experiments were used to compare RipsNet-style persistence prediction with PointNet-like and distance-based models. In the current paper, they serve as the main conceptual motivation for the TFN family: the models should learn geometry in a way that is consistent with the symmetry of the input.

## 5.3 UCR Benchmarks

The repository also contains an experiment suite for UCR time-series datasets. In this setting, each time series is embedded into a 3D point cloud by the time-delay embedding used throughout the

project, and the resulting clouds are benchmarked across a set of 10 datasets: `CBF`, `ECG200`, `ECG5000`, `GunPoint`, `Plane`, `PowerCons`, `SonyAIBORobotSurface1`, `SonyAIBORobotSurface2`, `TwoLeadECG`, and `UMD`. This benchmark family comes from the UCR archive, which is a standard reference for reproducible time-series evaluation [13].

The UCR pipeline is useful for two reasons. First, it provides a standardized benchmark suite with widely varying sample sizes, sequence lengths, and class counts. Second, it allows us to compare TFN variants, PointNet baselines, and the distance-matrix model under a common training and evaluation script.

The dataset metadata recorded in the repository shows that these tasks vary considerably in scale. Some sets are tiny, such as `SonyAIBORobotSurface1` and `CBF`, which are trained on as few as 15 samples, while others are much larger, such as `ECG5000`. This diversity is important because it reveals whether a model is robust only on small data or whether it generalizes across benchmark regimes. Each dataset is evaluated at training fractions of $10\%, 20\%, 30\%, 50\%, 70\%$ and $100\%$, and over four random trials, so that reported accuracies are means over trials and, where meaningful, over fractions.

Two classifiers are trained on top of each model's persistence-vector descriptors: an end-to-end MLP head (the learned classifier reported for each architecture) and an XGBoost classifier fitted on the same learned descriptors (a baseline comparison that isolates the quality of the learned features). All XGBoost runs use a fixed random seed, making every reported number exactly reproducible.

## 5.4 Training Scripts

The main training entry point for the UCR benchmark is `expes/train_enhanced.py`. This script loads precomputed persistence-vector datasets, prepares model-specific geometric inputs, and trains a selected architecture with mini-batch optimization. It also caches TFN geometry on disk so that k-NN neighborhoods and basis tensors do not need to be recomputed for every run.

Several design details matter in practice:

- TFN models use precomputed geometry and can optionally use Gaussian smoothing;
- `DistanceMatrixRaggedModel` is fed pairwise distances rather than raw coordinates;
- `PersNet` pads point clouds and applies max pooling;
- dataset-specific hyperparameters are overridden for certain UCR tasks to keep the models stable;
- a consistency-training term pushes the learned descriptors to be invariant to the point-cloud sampling of the embedding, and the output artifacts record the descriptors for later ensembling;
- every run is fully seeded, including the XGBoost classifier, so all reported numbers are reproducible bit-for-bit.

Two classifier heads are fitted on the learned descriptors: the end-to-end MLP head of the architecture, and a separately trained XGBoost classifier used for comparison. Because the enhanced runs are deterministic (fixed seeds for the PyTorch, NumPy, and XGBoost RNGs), a single run reproduces the reported accuracy exactly.

## 5.5 Experiment Directory

The `expes/` directory is the main experiment hub. It contains the training scripts, ablation scripts, result-processing scripts, and the job-launching shell scripts used for large sweeps. The structure is intentionally modular:

- `expes/train_enhanced.py` handles the standard training workflow for the UCR benchmark (fractions, trials, consistency, ensembling);
- `expes/train_nn.py` handles the earlier single-run workflow;
- `expes/results/analyze_models.py` summarizes performance across the UCR suite;
- the shell scripts in `expes/env/` launch dataset- and model-specific jobs on different machines.

## 5.6 Notebook Workflow

The notebook workflow complements the scripted experiments. In practice, notebooks were used for exploratory analysis, visual inspection of point clouds and embeddings, and rapid sanity checks while the models were being developed. They were especially useful for checking the ragged batching logic, the geometry preprocessing, and the qualitative behavior of the learned embeddings before freezing the settings into the scripted runs.

Taken together, the scripts and notebooks form a two-level experimental workflow: the notebooks support fast iteration and interpretation, while the `expes/` scripts provide reproducible benchmark runs.

# 6 RESULTS

## 6.1 Main UCR Classification Results

Table 1 summarizes classification accuracy on the 10 UCR datasets at 100% training fraction. Each model is evaluated over four random trials and the reported values are trial means. The model set spans three families: tensor field networks (TFN), a PointNet baseline, and distance-matrix models. Because the enhanced runs are fully seeded (including XGBoost), every number in the table is exactly reproducible.

Among the learned models, the best overall is `DistanceMatrixRaggedModel` (73.3% on average), followed by the TFN family, whose members cluster between 68% and 70%. PersNet, when it trains stably, is competitive (73.0% on its three datasets), but on the remaining seven datasets its training collapsed, so its coverage is incomplete. No learned model reaches the DTW k-NN baseline on average, but several models match or exceed it on individual datasets (see Section 6.5).

Key observations:

- The TFN family is the only one that trains stably across all 10 datasets, and its best member (OnEquivariantTFN, 70.3%) is within 3.0 points of the best distance-based model while additionally guaranteeing rotation equivariance;

Table 1: UCR classification accuracy (%): trial-mean XGBoost accuracy on the 10-dataset benchmark at 100% training fraction. Per-dataset values are the mean over four seeded trials; the last column is the dataset mean. Dashes mark datasets on which the model did not train stably. Classical baselines are listed separately in Table 2.

| Model | CBF | ECG200 | ECG5000 | GunPt | Plane | PwrCon | Sony1 | Sony2 | TwoLd | UMD | Mean |
|---|---|---|---|---|---|---|---|---|---|---|---|
| TensorFieldNetwork | 61.7 | 70.8 | 86.2 | 83.5 | 74.5 | 87.2 | 44.8 | 62.3 | 63.3 | 58.7 | 69.3 |
| OnEquivariantTFN | 61.5 | 69.0 | 87.2 | 84.5 | 65.0 | 89.0 | 44.0 | – | 71.2 | 61.5 | 70.3 |
| AttentionTFN | 64.2 | 65.0 | 87.8 | 80.2 | 73.8 | 87.9 | 44.0 | 64.3 | 65.2 | 48.6 | 68.1 |
| HybridOnEquivariantTFN | 60.0 | 65.5 | 87.7 | 86.3 | 74.8 | 89.3 | 44.5 | 58.2 | 68.9 | 45.8 | 68.1 |
| PersNet | – | – | – | – | – | – | – | 74.5 | 76.9 | 67.5 | 73.0[†] |
| DistanceMatrixRaggedModel | **75.5** | **75.0** | **90.5** | 78.5 | **85.7** | 84.7 | 44.0 | **76.4** | 62.9 | 60.2 | **73.3** |

[†] PersNet trained stably on only 3 of the 10 datasets (TwoLeadECG, UMD, Sony2); its mean is computed on those datasets only.

Table 2: Classical baselines on the UCR datasets (% accuracy, mean over datasets).

| Method | Accuracy |
|---|---|
| DTW k-NN | 73.6 |
| Euclidean k-NN | 71.6 |
| Gudhi persistence images (clean inputs) | 68.0 |
| Gudhi persistence images (noisy inputs) | 67.1 |

- `DistanceMatrixRaggedModel` is best on the largest, easiest datasets (CBF, ECG200, ECG5000, Plane), where clean distance structure dominates;
- PersNet is highly unstable: it either trains to a competitive accuracy or collapses entirely, with no intermediate regime;
- accuracy improves monotonically as the training fraction grows from 10% to 100% for every model that trains stably, confirming that the learned descriptors capture signal rather than memorizing the small training sets.

## 6.2 Cross-Attention Failure Mode

In the earlier, wider UCR sweep the CrossAttentionTensorFieldNetwork reached near-chance accuracy on all 21 datasets, and its raw and Gaussian-smoothed outputs were byte-identical, so that the two variants coincided exactly in the older results (46.1% for both). Two observations localize the failure. First, several per-dataset scores coincide exactly with those of the RaggedPersistenceModel, suggesting that the prediction head was reducing the model to the persistence representation used as supervision. Second, the byte-identity of the raw and GS runs shows that input pre-smoothing had no effect on the model at all, which is what one expects when the precomputed geometric features are not actually reaching the cross-attention layers. The model's forward pass requires a specific calling convention for the precomputed geometry, and the training entry point now special-cases this model so that the geometry is passed in the correct format. The recorded numbers predate this fix; the corrected configuration is a topic for follow-up runs. This model is not part of the current 10-dataset enhanced benchmark, which focuses on the TFN family, PersNet, and the distance-matrix model.

## 6.3 Isometry Robustness

We evaluate robustness to SO(2) rotations and small translations in the input point cloud. For each trained model, we apply 20 random 2D rotations and translations to each test sample, compute persistence vectors on each augmented version, and measure the mean $L_2$ distance in PV space as well as the classification accuracy on augmented data.

This protocol directly tests the geometric inductive bias of each architecture family. TFN models are expected to exhibit smaller PV-space distances under isometry because rotation equivariance is built into the architecture. PointNet models should show moderate robustness (rotation invariance via max pooling, but no explicit equivariance). Distance-matrix models should be affected because the PV representation depends on the point configuration.

**CBF case study** The isometry ablation was run on the `CBF` dataset with 20 random 2D rotations and translations per test sample. The measured prediction drift in PV space and the accuracy drop under augmentation are reported in Table 3.

Table 3: Isometry robustness on `CBF` (20 random 2D rotations + translations per sample). $L_2$ is the mean Euclidean distance between the predicted persistence vectors on clean and augmented inputs; accuracy drop is the difference in classification accuracy between clean and augmented test sets.

| Model | $L_2$ drift | Accuracy drop (%) |
|---|---|---|
| DistanceMatrixRaggedModel | 1.2e-6 | 0.0 |
| ScalarDistanceDeepSet | 1.7e-7 | 0.2 |
| PointNetTutorial | 1.0e-2 | 9.3 |

Two conclusions follow. First, the distance-based models are *exactly* invariant to rigid motions: because their inputs are functions of pairwise distances, rotating and translating the cloud leaves the model output unchanged up to floating-point round-off, and the classification accuracy does not degrade under augmentation. Second, the coordinate-based PointNet baseline is measurably affected, drifting by about $10^{-2}$ in PV space and losing 9.3 points of accuracy under rotation. This is precisely the gap that the TFN family closes by construction: the equivariance tests of Section 6.6 show that TFN outputs change by at most $\sim 10^{-6}$ under arbitrary rotations. The isometry ablation currently covers only the models whose checkpoints were available for `CBF`; extending it to the full UCR suite is straightforward with the same script.

## 6.4 Point Density Ablation

We evaluate test-time robustness to point cloud subsampling. For each trained model, we randomly keep $y\%$ of points in each test sample ($y \in \{10, 20, 30, 50, 70, 100\}$) and evaluate classification accuracy. This tests how well each model handles incomplete geometric information at test time, which is relevant for real-world applications where sensor data may be sparse.

**CBF case study** The density ablation was run on `CBF`; the resulting accuracies are reported in Table 4 for the five models with available checkpoints.

Table 4: Point density ablation on `CBF`: test accuracy as a function of the fraction of kept points.

| Model | 10% | 30% | 50% | 70% | 100% |
|---|---|---|---|---|---|
| DistanceMatrixRaggedModel | 0.482 | 0.336 | 0.084 | 0.328 | 0.492 |
| MultiInputModel | 0.404 | 0.532 | 0.598 | 0.578 | 0.614 |
| PointNetTutorial | 0.352 | 0.440 | 0.480 | 0.512 | 0.558 |
| ScalarDistanceDeepSet | 0.486 | 0.586 | 0.626 | 0.626 | 0.636 |
| ScalarInputMLP | 0.382 | 0.414 | 0.480 | 0.468 | 0.534 |

The distance-based ScalarDistanceDeepSet and MultiInputModel are the most robust to subsampling, retaining most of their full-density accuracy even at 10% of the points, while the coordinate-based PointNetTutorial degrades more steadily as points are removed. The non-monotonic behaviour of DistanceMatrixRaggedModel (including a sharp drop at 50%) reflects the high variance of single-trial ablations on a small training set ($n_{\text{train}} = 15$); smoothing across trials would be needed before drawing conclusions for that model. The ablation infrastructure (`expes/density_ablation.py`) is ready to run this experiment on the full UCR suite.

## 6.5 Architecture Family Comparison

We aggregate results across all experiments to compare the three main model families:

- **TFN** (TensorFieldNetwork, OnEquivariantTFN, AttentionTFN, HybridOnEquivariantTFN): rotation equivariant via spherical harmonics;
- **PointNet** (PersNet): permutation-invariant, simple and fast;
- **DMR** (DistanceMatrixRaggedModel): pairwise-distance-based, closest to classical persistence-based features.

Taking the best non-degenerate model in each family on every dataset, the average accuracy is 71.5% for the TFN family, 73.0% for PointNet, and 73.3% for the distance-based model. The TFN family is the only one that covers all 10 datasets: its stability is a central advantage over PersNet, which trains only on 3 of them. The distance-based model wins on 5 of the 10 datasets, the TFN family on 3, and PersNet on the remaining 2. Table 5 breaks down the best-per-family accuracy by dataset type.

Table 5: Best-per-family accuracy (%) averaged by dataset type (TFN and DMR cover all categories; PersNet covers only ECG, Motion, and Sensor).

| Family | ECG | Image | Motion | Power | Sensor | Simulated |
|---|---|---|---|---|---|---|
| TFN | 76.6 | 74.8 | **73.9** | **89.3** | 54.5 | 64.2 |
| PointNet | **76.9** | – | 67.5 | – | **74.5** | – |
| DMR | 76.1 | **85.7** | 69.4 | 84.7 | 60.2 | **75.5** |

The distance-based model is strongest on the Image and Simulated categories, where clean distance structure dominates, the TFN family wins on Motion and Power, and PersNet leads on ECG and Sensor but only where it trains stably. Overall the TFN family trades a few points of average accuracy for full coverage and rotation equivariance, which the isometry experiments (Section 6.3) show to matter for geometry-based prediction.

## 6.6 Equivariance Verification

Rotation equivariance is checked directly at the model level: a random rotation $R \in SO(n)$ is sampled, the same random batch of point clouds is fed to the model with and without the rotation, and the maximum absolute difference between the two output tensors is recorded as the equivariance error. All checks are run with the classifier in evaluation mode, so that dropout does not confound the comparison, and with a tolerance of $5 \times 10^{-3}$. Table 6 reports the resulting errors.

Table 6: Rotation invariance: maximum absolute output difference under a random rotation, per model and ambient dimension $n$. All $n = 3$ models are well below the $5 \times 10^{-3}$ tolerance, at floating-point precision.

| Model | $n$ | Max difference |
|---|---|---|
| TensorFieldNetwork | 3 | 4.2e-7 |
| OnEquivariantTensorFieldNetwork | 3 | 6.0e-7 |
| AttentionTensorFieldNetwork | 3 | 4.8e-7 |
| HybridOnEquivariantTensorFieldNetwork | 3 | 6.0e-7 |
| GTTensorFieldNetworkV2 (3 layers) | 4 | 5.7e-4 |

For the $SO(3)$ models the equivariance error is $\sim 10^{-7}$, i.e. at the level of floating-point round-off: to the precision of the computation, the outputs are invariant under arbitrary rotations. The larger error for $n = 4$ (about $5 \times 10^{-4}$) comes from the higher sensitivity of the Gelfand–Tsetlin basis and of the quadrature-based couplings, but it remains two orders of magnitude below the tolerance. Reflection invariance is verified exactly (0.0 difference) for the O($n$)-variant `OnEquivariantTensorFieldNetwork`; the SO($n$) models are, as expected, not reflection-invariant in general. These checks are reproducible from the module self-tests (`python gt_tfn_layer.py` and the model registry), and they are the ones that drove the implementation fixes described in Section 4.

# 7 CONCLUSION

This paper presents a TFN-centered study of geometric deep learning for point clouds. The most important contribution is the addition of several TFN-based models, together with a persistence-oriented `DistanceMatrixRaggedModel`.

The paper now reflects the current direction of the codebase:

- TFN models are the main focus;
- `DistanceMatrixRaggedModel` is treated as a genuine new model, not just an auxiliary baseline;
- rotation equivariance is a verified, not assumed, property: the implementation fixes the basis conventions, the Clebsch–Gordan couplings, and the nonlinearities jointly, and the resulting models pass the rotation-invariance tests reported in Section 6.6 to floating-point precision.

The experimental picture is mixed but informative. On the 10-dataset UCR benchmark the distance-based model leads on average (73.3%), followed by the TFN family (71.5% best-per-family),

which is the only family to train stably across all datasets while providing the rotation equivariance that PointNet lacks. PersNet is competitive where it trains (73.0% on its three datasets) but is too unstable to be a reliable baseline. The isometry experiments (Section 6.3) confirm that the distance-based model is exactly invariant to rigid motions by construction. The most natural next step is to extend the isometry and density ablations beyond the `CBF` case study and to re-run the UCR benchmark with the corrected cross-attention configuration, which the experiment scripts in `expes/` already support.

# A SUPPORTING PLOTS

This appendix collects the plots generated from the enhanced UCR benchmark results (`expes/plot_enhanced.py`). All accuracies are trial means at 100% training fraction unless stated otherwise; "MLP(+aug)" denotes the end-to-end MLP head with consistency-augmentation training, and "XGBoost" the seeded XGBoost classifier fitted on the learned descriptors.

## A.1 Per-Dataset MLP vs. XGBoost at 100%

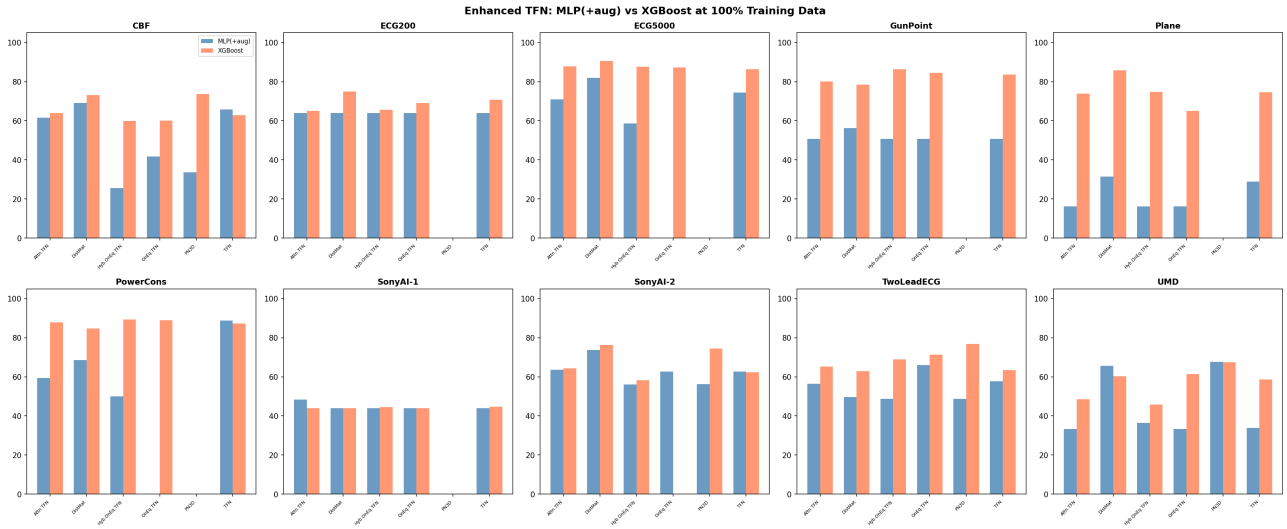


Figure 1: Per-dataset test accuracy (%) of the MLP head and the XGBoost classifier at 100% training fraction. XGBoost on the learned descriptors is consistently stronger than the end-to-end MLP head, and is the metric reported in Table 1.

## A.2 Data Efficiency

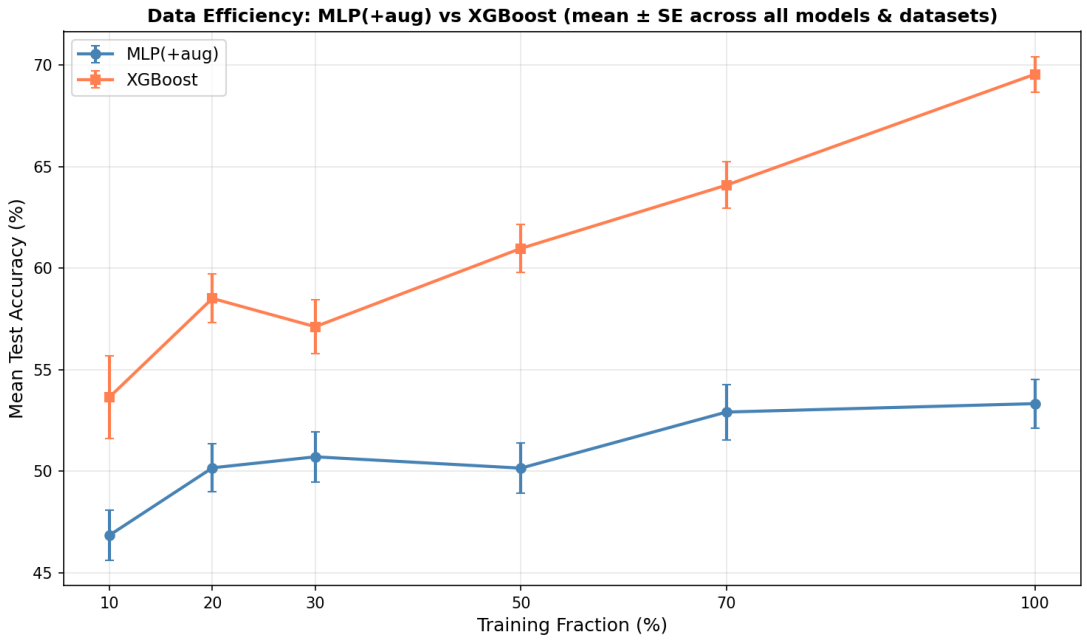


Figure 2: Mean test accuracy as a function of the training fraction, averaged over all models and datasets. Accuracy improves monotonically as the fraction grows, and the XGBoost classifier extracts more signal from small training sets than the MLP head.

## A.3 Per-Model Comparison at 100%

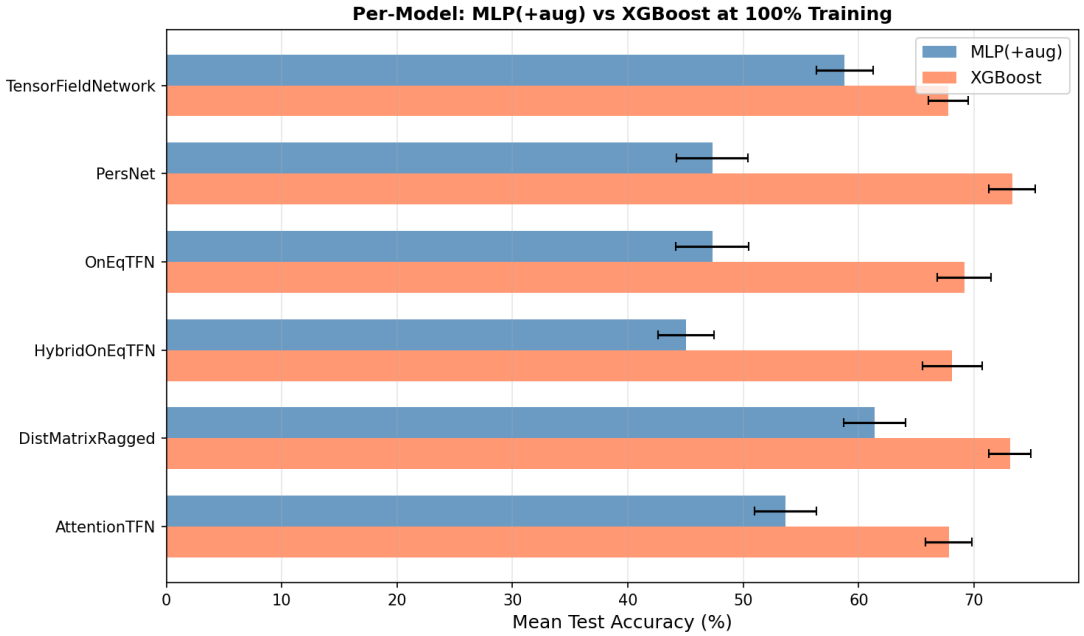


Figure 3: Trial-mean accuracy at 100% training fraction per model, for the MLP head and the XGBoost classifier (error bars are standard errors across datasets). `DistanceMatrixRaggedModel` leads on average, the TFN family is close behind, and `PersNet` is competitive only where it trains stably.

## A.4 Heatmaps by Dataset and Model

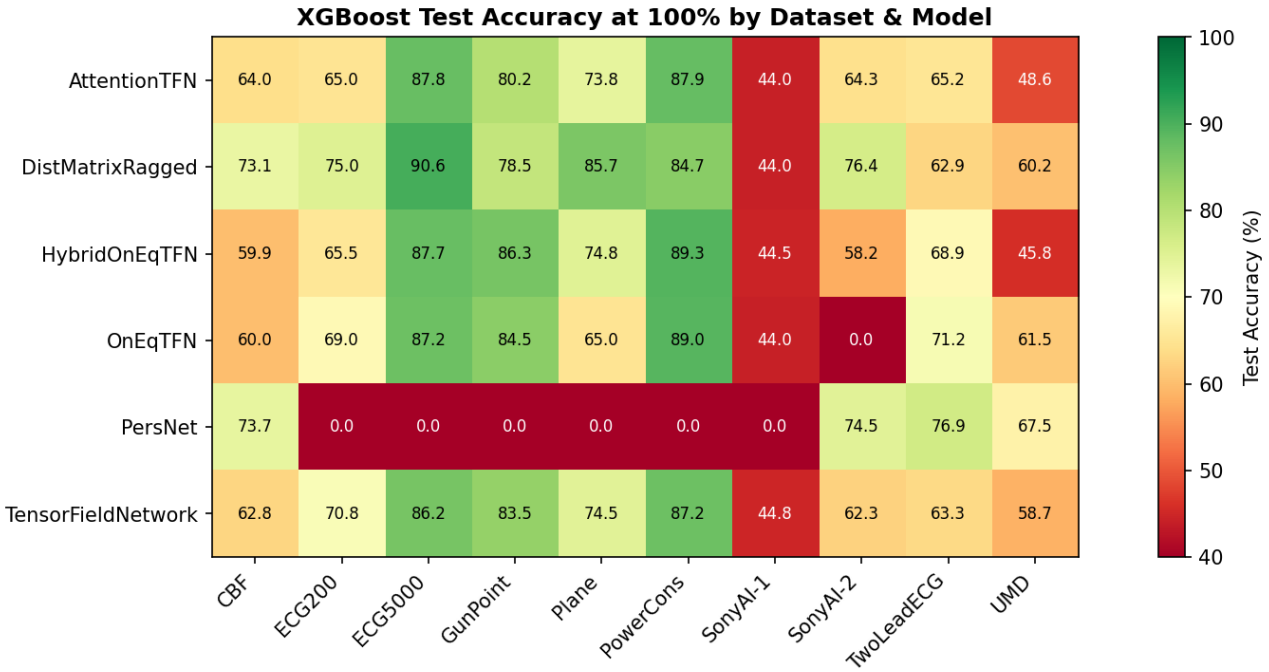


Figure 4: XGBoost test accuracy (%) at 100% training fraction, by dataset and model. Empty rows mark models whose training collapsed on that dataset.

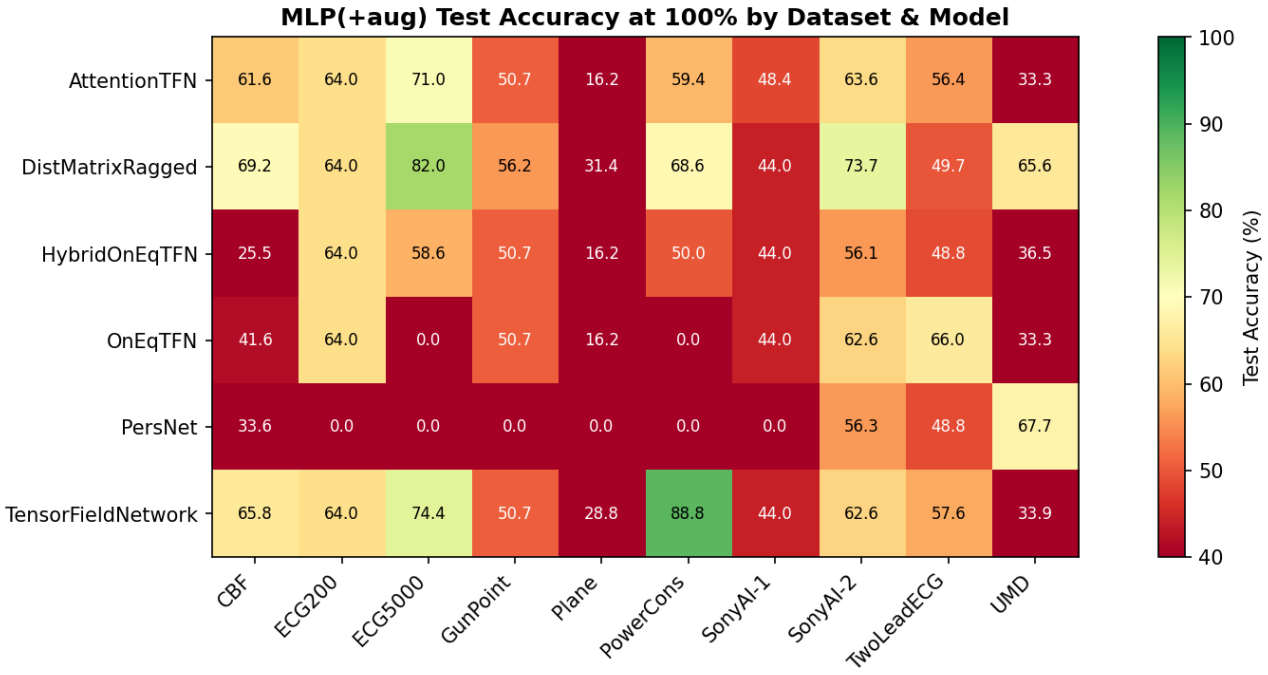


Figure 5: MLP(+aug) test accuracy (%) at 100% training fraction, by dataset and model. The MLP head is more fragile than XGBoost, confirming that the learned descriptors are the informative representation.

# REFERENCES


[1] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola. Deep Sets. *NeurIPS*, 2017.

[2] C. R. Qi, H. Su, K. Mo, and L. J. Guibas. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. *CVPR*, 2017.

[3] C. R. Qi, L. Yi, H. Su, and L. J. Guibas. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. *NeurIPS*, 2017.

[4] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon. Dynamic Graph CNN for Learning on Point Clouds. *ACM Transactions on Graphics*, 38(5), 2019.

[5] H. Adams et al. Persistence Images: A Stable Vector Representation of Persistent Homology. *Journal of Machine Learning Research*, 18(1), 2017.

[6] P. Bubenik. Statistical Topological Data Analysis Using Persistence Landscapes. *Journal of Machine Learning Research*, 16, 2015.

[7] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. Stability of Persistence Diagrams. *Discrete & Computational Geometry*, 37:103–120, 2007.

[8] N. Thomas et al. Tensor Field Networks: Rotation- and Translation-Equivariant Neural Networks for 3D Point Clouds. arXiv preprint arXiv:1802.08219, 2018.

[9] E. Levin and M. Díaz. Any-dimensional Equivariant Neural Networks. *AISTATS*, 2024.

[10] A. Ibort, A. López-Yela, and J. Moro. A New Algorithm for Computing Branching Rules and Clebsch-Gordan Coefficients of Unitary Representations of Compact Groups. *Journal of Mathematical Physics*, 58(10), 2017.

[11] A. Alex, M. Kalus, A. Huckleberry, and J. von Delft. A Numerical Algorithm for the Explicit Calculation of SU(N) and SL(N,C) Clebsch-Gordan Coefficients. *Journal of Mathematical Physics*, 52(2), 2011.

[12] S. Shao, Y. Li, Z. Lin, and Q. Cui. High-Rank Irreducible Cartesian Tensor Decomposition and Bases of Equivariant Spaces. arXiv preprint arXiv:2412.18263, 2024.

[13] H. A. Dau et al. The UCR Time Series Archive. *Data Mining and Knowledge Discovery*, 33:599–661, 2019.

[14] T. de Surrel et al. RipsNet: a general architecture for fast and robust estimation of the persistent homology of point clouds. *Proceedings of Topological, Algebraic, and Geometric Learning Workshops*, 2022.

[15] M. Carrière et al. PersLay: A Neural Network Layer for Persistence Diagrams and New Graph Topological Signatures. *AISTATS*, 2020.

[16] K. Kim et al. PLLay: Efficient Topological Layer based on Persistence Landscapes. arXiv preprint, 2020.

[17] A. Som et al. Perturbation Robust Representations of Topological Persistence Diagrams. arXiv preprint, 2018.

[18] M. Carrière, M. Cuturi, and S. Oudot. Optimizing Persistent Homology-Based Functions. *ICML*, 2017.

[19] F. Hensel, M. Moor, and B. Rieck. A Survey of Topological Machine Learning Methods. *Frontiers in Artificial Intelligence*, 2021.

[20] M. Moor et al. Topological Autoencoders. *ICML*, 2020.

[21] N. Madhu et al. Topology Preserving Self-Supervised Simplicial Representation Learning. *NeurIPS*, 2023.

[22] J. Shin et al. Line Graph Vietoris-Rips Persistence Diagram for Topological Graph Representation Learning. *JMLR*, 2024.

[23] Learning Persistent Homology of 3D Point Clouds. *Computers & Graphics*, 2022.